Contents

Appendix — Synthetic Dataset Generator MVP

1. Purpose

Design a command-line tool that generates synthetic datasets for:

- LLM evaluation
- classifier training

- structured-output testing
- agent testing
- RAG evaluation
- robustness testing
- regression testing

The tool should generate **structured examples from a declarative specification**, rather than simply asking an LLM to "make 1,000 examples."

Core principle:

> **Synthetic data generation should be engineered as a controlled data-generation pipeline, not treated as unconstrained text generation.**

---

2. MVP Use Case

The initial MVP should generate datasets like the mortgage-question evaluation set.

A specification might define:

```yaml
dataset:
  name: mortgage_questions
  size: 1000

schema:
  fields:
    - question
    - intent
    - principal
    - annual_rate
    - term_years
    - payment
    - expected_outcome

categories:
  payment:
    weight: 0.20

  principal:
    weight: 0.20

  term:
    weight: 0.15

  rate:
    weight: 0.15

  clarification:
    weight: 0.15

  unsupported_scope:
    weight: 0.15
```

The generator produces JSONL:

```jsonl
{"question":"What is the payment on a $400,000 mortgage at 6.5% for 30 years?","intent":"payment",...}
```

The critical difference from ordinary text generation is that **the semantic structure of the dataset is specified first**.

---

3. Design Goals

The MVP should optimize for:

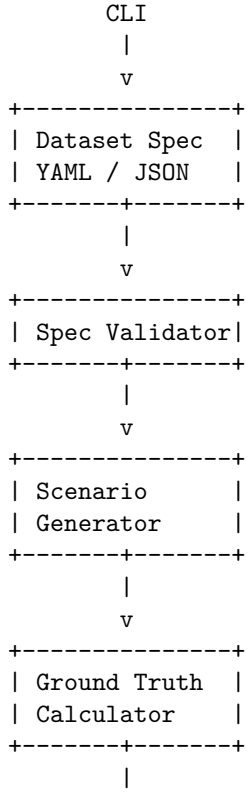
1. **Controllability**
2. **Reproducibility**
3. **Schema correctness**
4. **Distribution control**
5. **Variation**
6. **Validation**
7. **Traceability**
8. **Model independence**

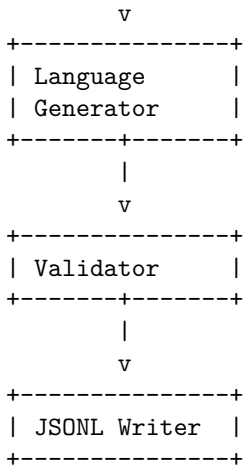
A generated dataset should be reproducible:

```
specification
      +
seed
      +
generator version
      ↓
same dataset
```

---

4. Architecture





An LLM is optional rather than fundamental to the architecture.

---

5. Separate Scenario Generation from Language Generation

This is the most important design decision.

Do **not** begin with:

```
"Generate 1,000 mortgage questions."
```

Instead generate an abstract scenario first:

```
{
  "intent": "payment",
  "principal": 500000,
  "annual_rate": 0.065,
  "term_years": 30,
  "expected_outcome": "calculated"
}
```

Then generate language representing that scenario:

```
"What would my monthly payment be on a
$500,000 mortgage at 6.5% for 30 years?"
```

This creates a clean separation:

```
Scenario
   |
   +-- Ground truth
   |
   +-- Natural-language realization
```

This dramatically improves dataset reliability.

---

6. Scenario Generator

The scenario generator produces valid structured situations.

For mortgage data:

```
principal
interest rate
term
payment
down payment
intent
expected outcome
```

It should support distributions:

```yaml
principal:
  distribution: lognormal
  min: 100000
  max: 2000000

annual_rate:
  distribution: uniform
  min: 0.03
  max: 0.09

term_years:
  values: [10, 15, 20, 25, 30]
```

The generator should be capable of producing both:

Valid scenarios

```
P > 0
r >= 0
n > 0
M sufficient to amortize
```

Deliberately invalid scenarios

```
payment <= first-period interest
```

The mortgage dataset explicitly includes such cases as `payment_too_low`.

---

7. Ground Truth Engine

Every generated scenario should have deterministic ground truth whenever possible.

For the mortgage example:

```
scenario
   ↓
mortgage calculator
   ↓
expected answer
```

For example:

```json
{
  "principal": 500000,
  "annual_rate": 0.065,
  "term_years": 30,
```

```
  "expected_payment": 3160.34
}
```

This is preferable to asking an LLM to generate both the question and its answer.

The general architecture becomes:

Generate parameters → calculate truth → generate language.

---

8. Natural-Language Generator

The generator converts structured scenarios into natural language.

There should be multiple generation strategies.

Template generation

```
"What is the monthly payment on a
{principal} mortgage at {rate} for {term}?"
```

Template variation

```
"How much would I pay each month if I borrowed
{principal} at {rate} over {term}?"
```

LLM generation

Give the LLM the structured scenario:

```
{
  "intent": "payment",
  "principal": 500000,
  "annual_rate": 0.065,
  "term_years": 30
}
```

and ask it to produce a natural user question representing exactly that scenario.

The LLM should **not be allowed to modify the ground truth**.

---

9. Controlled Variation

Synthetic data becomes useful when it captures linguistic diversity.

The generator should vary:

Numerical expression

```
$500,000
500000
500k
half a million dollars
```

Rate expression

```
6.5%
6.5 percent
an interest rate of 6.5%
at six and a half percent
```

Term expression

```
30 years
30-year mortgage
three decades
360 monthly payments
```

Question style

```
What is my payment?

How much would I pay each month?

What would the monthly payment come to?

How much would this mortgage cost me monthly?
```

This creates variation without losing semantic control.

---

10. Constraint System

The specification should allow constraints.

For example:

```yaml
constraints:
  - principal > 0
  - annual_rate >= 0
  - term_years in [10, 15, 20, 30]
```

More interesting constraints should allow relationships:

```yaml
constraints:
  - payment > principal * annual_rate / 12
```

and scenario-specific rules:

```yaml
payment_too_low:
  constraints:
    - payment <= principal * annual_rate / 12
```

This transforms the generator from a random-data script into a **constraint-driven synthetic data system**.

---

11. Dataset Schema

Every generated record should contain both the user-facing data and metadata.

For example:

```
{
  "id": "mortgage-000123",
  "input": {
    "question": "How much can I borrow if I can pay $3,000 a month at 6% for 30 years?"
  },
  "ground_truth": {
    "intent": "principal",
    "principal": null,
    "annual_rate": 0.06,
    "term_years": 30,
    "payment": 3000,
    "expected_outcome": "calculated"
  },
  "metadata": {
    "category": "principal",
    "generator": "template",
    "template_id": "principal_07",
    "seed": 183729
  }
}
```


The metadata is extremely valuable for debugging and evaluation.

---

12. Distribution Control

The tool should support explicit dataset distributions.

For example:

```
categories:
  payment: 25%
  principal: 20%
  term: 15%
  rate: 15%
  clarification: 10%
  unsupported_scope: 15%
```

The mortgage dataset currently has explicit category counts, including 5 payment, 5 principal, 5 term, 5 rate, 2 down-payment, 4 clarification, and 4 unsupported-scope examples.

A generator should allow the user to specify either:

`count`

or:

`percentage`

for each category.

---

13. Negative and Edge Cases

A serious synthetic-data generator must deliberately generate difficult cases.

Categories could include:

```
normal
boundary
invalid
ambiguous
underspecified
unsupported
adversarial
```

For the mortgage example:

```
zero interest
very high interest
very short term
very long term
payment equal to interest
payment below interest
missing rate
missing term
ambiguous rate
adjustable-rate mortgage
property taxes
HOA
insurance
```

The existing mortgage dataset explicitly uses clarification cases and unsupported-scope cases to test these behaviors.

---

14. LLM-as-Generator vs Template Generator

The MVP should support both.

Templates

Advantages:

- deterministic
- inexpensive
- reproducible
- easy to validate

Disadvantages:

- limited linguistic diversity

LLM generation

Advantages:

- much greater linguistic diversity
- realistic phrasing
- paraphrases
- conversational language

Disadvantages:

- probabilistic
- potentially invalid
- potentially changes semantics

- higher cost

Therefore:

> **Templates should establish correctness; LLMs should provide diversity.**

---

15. Semantic Validation

Every generated example should be validated.

A validator should check:

```
Does the question contain the intended parameters?
Does it express the intended intent?
Does it preserve the scenario?
Does it request the intended operation?
Does it remain within scope?
```

For LLM-generated examples, the validation pipeline could be:

```
Scenario
   |
   v
LLM generates question
   |
   v
LLM/parser extracts scenario
   |
   v
Compare extracted scenario
   |
   +-- match → accept
   |
   +-- mismatch → reject/regenerate
```

This is a particularly useful example of **generation followed by verification**.

---

16. Deduplication

Synthetic generators tend to produce duplicates.

The MVP should support at least:

Exact deduplication

Normalize text and hash it.

Near deduplication

Compare embeddings or normalized representations.

For example:

```
"What is the payment on $500k at 6% for 30 years?"

"What would the monthly payment be for a
$500,000 mortgage at 6% over 30 years?"
```

These are semantically almost identical.

Whether they should both remain depends on the dataset's purpose.

---

17. Quality Metrics

The CLI should produce a dataset report.

For example:

```
Dataset: mortgage_questions
Records: 10,000

Category distribution
---------------------
payment             2,000
principal           2,000
term                1,500
rate                1,500
clarification       1,000
unsupported_scope   2,000

Validation
----------
valid                9,842
rejected               158

Duplicates
----------
exact                    3
near                   127

LLM generation
--------------
generated            7,500
template             2,500

Average generation cost: $...
```

The generator should not merely produce a file; it should tell the engineer whether the resulting dataset is healthy.

---

18. Reproducibility

Every dataset should record:

```
generator version
specification hash
random seed
model name
model version
generation parameters
timestamp
```

For example:

```
{
  "generator_version": "0.1.0",
  "spec_hash": "a84f...",
  "seed": 42197,
  "model": "qwen3...",
  "temperature": 0.7
}
```

This allows a dataset to be regenerated or investigated later.

---

19. CLI Design

A simple command structure:

```
synthgen generate mortgage.yaml
```

Options:

```
synthgen generate mortgage.yaml \
    --size 10000 \
    --seed 42 \
    --output mortgage.jsonl
```

Validation:

```
synthgen validate mortgage.jsonl
```

Statistics:

```
synthgen stats mortgage.jsonl
```

Preview:

```
synthgen preview mortgage.yaml --count 20
```

Reproduce:

```
synthgen reproduce dataset-manifest.json
```

A particularly useful command:

```
synthgen inspect mortgage.jsonl
```

which could show category distributions, failures, duplicates, and sample records.

---

20. Suggested Project Structure

```
synthgen/
|
+-- pyproject.toml
+-- README.md
|
+-- src/
|   +-- synthgen/
|       +-- cli.py
|       +-- spec.py
|       +-- schema.py
```

```
|       +-- scenarios.py
|       +-- distributions.py
|       +-- constraints.py
|       +-- generators.py
|       +-- templates.py
|       +-- llm.py
|       +-- validators.py
|       +-- dedup.py
|       +-- metrics.py
|       +-- writers.py
|
+-- tests/
|   +-- test_spec.py
|   +-- test_scenarios.py
|   +-- test_constraints.py
|   +-- test_generators.py
|   +-- test_validation.py
|   +-- test_dedup.py
|
+-- examples/
    +-- mortgage.yaml
```

---

21. Development Sequence

Phase 1 — Deterministic generator

Implement:

- specification parser
- random scenario generation
- constraints
- JSONL output
- seeds

Phase 2 — Ground truth

Add:

- deterministic calculators
- expected outcomes
- validation

Phase 3 — Templates

Add:

- template library
- controlled linguistic variation

Phase 4 — LLM generation

Add:

- model adapter
- structured generation
- regeneration on failure

Phase 5 — Quality system

Add:

- deduplication
- statistics
- dataset reports
- quality thresholds

Phase 6 — CLI polish

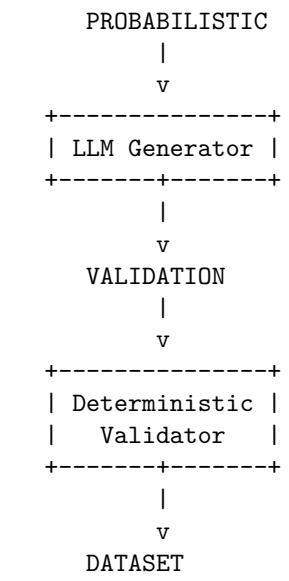
Add:

- configuration
- progress reporting
- caching
- manifests
- reproducibility

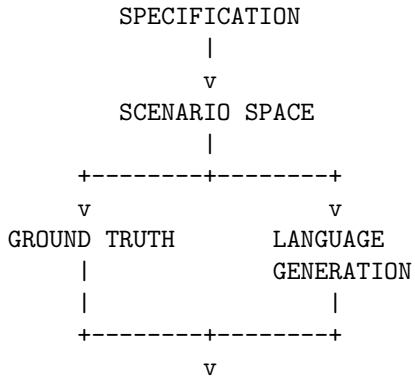
---

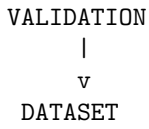
22. The Key Architectural Insight

The synthetic-data generator itself should follow the same architecture as the hybrid mortgage calculator:



But there is an even more important pattern:





This makes the generator a miniature **data engineering + AI evaluation platform** rather than a prompt wrapper.

23. Bootcamp Learning Objective

This appendix would fit particularly well after the chapters on **testing AI systems, evaluation, specification engineering, and coding agents**.

The project teaches a powerful general lesson:

> **Synthetic data should be generated from a specification of the space you want to test, not from a request for examples.**

The specification defines the semantic space.

The deterministic generator defines ground truth.

The LLM provides linguistic diversity.

The validator enforces correctness.

The resulting dataset becomes an engineering artifact that can itself be versioned, tested, measured, and reproduced.

That is the transition from **"generate some test data"** to **synthetic dataset engineering**.